

## Relatório: Padrões de Teste Aplicados

### Padrões de Criação de Dados (Builders)

Por que CarrinhoBuilder vs CarrinhoMother?

O CarrinhoBuilder foi escolhido em vez de um CarrinhoMother porque oferece maior flexibilidade na criação de cenários de teste. Enquanto o Object Mother é ótimo para objetos imutáveis com estados predefinidos (como vemos no UserMother), o Builder é mais adequado quando precisamos de diferentes combinações de estados.

Exemplo Antes e Depois

Antes (Setup Manual):

```
const user = new User('user-1', 'João Silva', 'joao@email.com', 'PREMIUM');
const item1 = new Item('Produto 1', 120);
const item2 = new Item('Produto 2', 80);
const itens = [item1, item2];
const carrinho = new Carrinho(user, itens);
```

```
const userPadrao = new User('user-2', 'Maria', 'maria@email.com', 'PADRAO');
const outroCarrinho = new Carrinho(userPadrao, [new Item('Item X', 50)]);
```

Depois (Com Builder):

```
const carrinhoPremium = new CarrinhoBuilder()
  .comUser(UserMother.umUsuarioPremium())
  .comItens([
    new Item('Produto 1', 120),
    new Item('Produto 2', 80)
  ])
  .build();
```

```
const carrinhoSimples = CarrinhoBuilder.umCarrinhoPadrao();
const carrinhoVazio = new CarrinhoBuilder().vazio().build();
```

Benefícios para Legibilidade e Manutenção

Encapsulamento de Complexidade: O Builder esconde a complexidade da criação do objeto, expondo apenas uma interface fluente e expressiva.

Valores Padrão: Reduz a necessidade de especificar todos os atributos em cada teste.

Reutilização: Métodos como vazio() e umCarrinhoPadrao() podem ser reutilizados em vários testes.

Flexibilidade: Fácil adição de novos cenários sem duplicar código.

### Padrões de Test Doubles (Mocks vs. Stubs)

Analisando o teste de sucesso para cliente Premium:

Identificação dos Test Doubles

```
const gatewayStub = {  
  cobrar: jest.fn().mockResolvedValue({ success: true })  
};
```

```
const emailMock = {  
  enviarEmail: jest.fn().mockResolvedValue(undefined)  
};
```

Por que Gateway é Stub?

O GatewayPagamento é um stub porque nos interessa apenas seu retorno ({ success: true }) para prosseguir com o fluxo do teste

A verificação toHaveBeenCalledWith(180, cartaoCredito) é secundária, apenas para validar o valor do desconto

O foco principal é o estado (resultado da operação) e não o comportamento

Por que EmailService é Mock?

O EmailService é um mock porque o foco é verificar como ele foi chamado

Não há retorno significativo do envio de email que afete o estado do sistema

O importante é verificar o comportamento: se foi chamado uma vez e com os parâmetros corretos

Validamos a iteração usando toHaveBeenCalledTimes(1) e toHaveBeenCalledWith(...)

## Conclusão: Impacto na Sustentabilidade dos Testes

O uso consciente destes padrões contribui para uma suíte de testes mais sustentável:

Builders:

Reduzem duplicação de código, centralizam a lógica de criação de objetos, facilitam manutenção quando a estrutura dos objetos muda

Test Doubles: Isolam o código sob teste, tornam explícito o papel de cada dependência, facilitam a identificação do que está sendo realmente testado

Prevenção de Test Smells:

Evita teste frágil (através de builders reutilizáveis), reduz acoplamento com dependências externas, torna os testes mais legíveis e mantíveis

O uso destes padrões não é apenas uma questão de organização, mas uma estratégia deliberada para criar testes que sejam tão sustentáveis quanto o código de produção que eles verificam.